

LAPORAN TUGAS 5

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Jhodi Andika (2410651050)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

DAFTAR ISI

BAB III	3
PEMBAHASAN	3
3.1 Analisis Masalah.....	3
3.2 Implementasi	4
3.2.1 Dining deadlock.....	4
3.2.2 Resource ordering	6
3.2.3 Limiting philosophers.....	8
3.3Testing & Output	10
3.3.1 Deadlock Demonstration	10
3.3.2 Resource Ordering	11
3.3.3 Solution_Limiting	12
3.4Analisis Hasil.....	14
3.5 Tabel perbandingan efisiensi dan fairness	15
3.6 Analisis mendalam:	16
BAB IV.....	17
PENUTUP.....	17
4.1 Kesimpulan.....	17

BAB III

PEMBAHASAN

DINING PHILOSOPHERS PROBLEM (Akhiran NIM Genap)

Deskripsi Masalah:

Implementasikan solusi untuk masalah klasik 5 filsuf makan dengan **mencegah deadlock!**

Kondisi:

- 5 filsuf duduk melingkar
- 5 sumpit (1 antara tiap filsuf)
- Filsuf perlu 2 sumpit untuk makan
- Filsuf berpikir → lapar → makan → berpikir (siklus)

Requirements:

Deadlock Demonstration:

1. Buat program yang **MENIMBULKAN DEADLOCK**
2. Semua filsuf mengambil sumpit kiri dulu, lalu kanan
3. Tampilkan saat program hang/deadlock
4. Jelaskan mengapa deadlock terjadi

Deadlock Prevention:

1. Implementasikan **minimal 2 solusi berbeda** untuk mencegah deadlock:
 - Solusi A: Resource Ordering (filsuf genap ambil kiri dulu, ganjil ambil kanan dulu)
 - Solusi B: Limiting Philosophers (max 4 filsuf makan bersamaan)
 - Solusi C: Asymmetric Solution (1 filsuf urutan berbeda)
2. Jalankan masing-masing solusi selama 60 detik
3. Bandingkan **efisiensi** (berapa kali masing-masing filsuf berhasil makan)

3.1 Analisis Masalah

Dining Philosophers Problem adalah sebuah ilustrasi klasik dalam studi sinkronisasi dan koordinasi akses sumber daya dalam sistem operasi. Pada skenario ini, terdapat lima filsuf yang duduk mengelilingi meja bundar. Di antara setiap dua filsuf, terdapat satu buah sumpit. Dengan demikian, terdapat lima sumpit yang tersedia untuk kelima filsuf tersebut.

Setiap filsuf secara bergantian melakukan tiga aktivitas: berpikir, merasa lapar, dan makan. Ketika berpikir, filsuf tidak memerlukan sumpit. Namun, ketika merasa lapar, filsuf perlu mengambil dua

sumpit (yang berada di kiri dan kanannya) untuk dapat makan. Setelah selesai makan, filsuf akan meletakkan kembali kedua sumpit tersebut dan kembali ke keadaan berpikir.

Permasalahan deadlock dapat timbul ketika semua filsuf secara bersamaan mengambil sumpit di sebelah kiri mereka. Akibatnya, setiap filsuf memegang satu sumpit dan menunggu sumpit di sebelah kanannya yang sedang dipegang oleh filsuf lain. Situasi ini menyebabkan kebuntuan karena tidak ada filsuf yang dapat memperoleh kedua sumpit untuk makan.

Untuk mengatasi deadlock ini, setidaknya terdapat dua solusi yang dapat diimplementasikan. Pertama, Resource Ordering (pengurutan sumber daya) di mana filsuf diharuskan mengambil sumpit dengan urutan tertentu, misalnya berdasarkan nomor sumpit (ganjil-genap). Kedua, Limiting Philosophers (pembatasan filsuf) yang membatasi hanya sejumlah tertentu filsuf (misalnya empat dari lima) yang boleh mencoba makan secara bersamaan. Ketiga, Asymmetric Solution (solusi asimetris) di mana satu filsuf mengambil sumpit dengan urutan yang berbeda dari yang lain.

Untuk mengevaluasi efektivitas solusi, setiap solusi dijalankan selama 60 detik dan dihitung berapa kali masing-masing filsuf berhasil makan. Dengan demikian, dapat dibandingkan performa dan keadilan dari setiap solusi.

3.2 Implementasi

3.2.1 Dining deadlock

Algoritma

Inisialisasi:

- Inisialisasi 5 mutex (sumpit) dan 5 thread (filsuf).
- Setiap filsuf memiliki ID (1-5) dan dua sumpit: kiri (sesuai ID) dan kanan ($ID+1$, modulo 5).

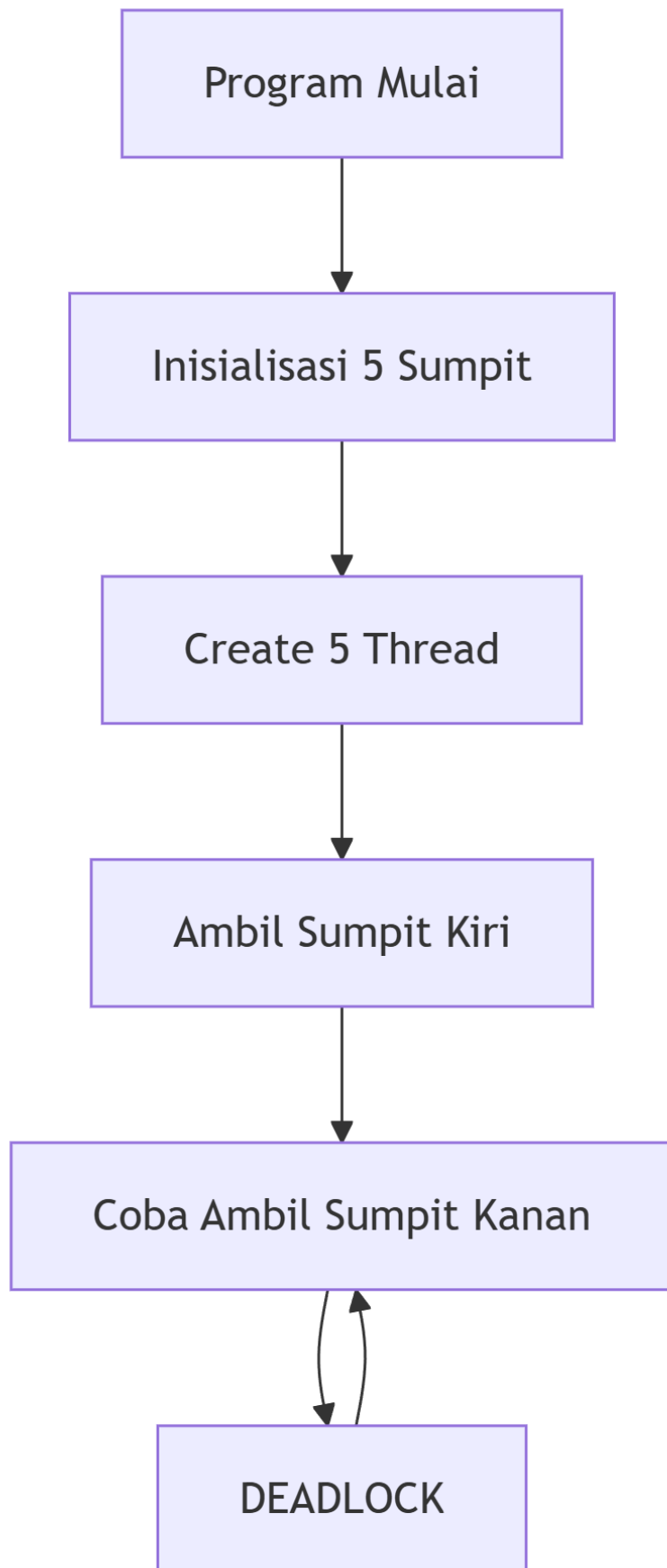
Perilaku Filsuf:

- Filsuf mengambil sumpit kiri terlebih dahulu.
- Kemudian, filsuf mencoba mengambil sumpit kanan.
- Jika berhasil mengambil kedua sumpit, filsuf makan lalu melepaskan kedua sumpit.
- Namun, dalam desain ini, setelah mengambil sumpit kiri, semua filsuf akan mencoba mengambil sumpit kanan secara bersamaan, yang sudah dipegang oleh filsuf lain, sehingga terjadi deadlock.

Deadlock:

- Kondisi deadlock terjadi karena setiap filsuf memegang satu sumpit dan menunggu sumpit yang dipegang oleh filsuf lain, sehingga tidak ada filsuf yang dapat melanjutkan.

Flochart



3.2.2 Resource ordering

Algoritma Resource Ordering

Inisialisasi:

- Inisialisasi 5 mutex (sumpit) dan 5 thread (filsuf).
- Setiap filsuf memiliki ID (1-5) dan dua sumpit: kiri (sesuai ID) dan kanan (ID+1, modulo 5).

Perilaku Filsuf dengan Resource Ordering:

- Pengambilan sumpit didasarkan pada penomoran, bukan posisi.
- Urutan pengambilan: nomor kecil dulu, kemudian nomor besar.
- Setelah makan, sumpit dilepaskan dalam urutan terbalik.
- Aturan ini mencegah circular wait dan deadlock.

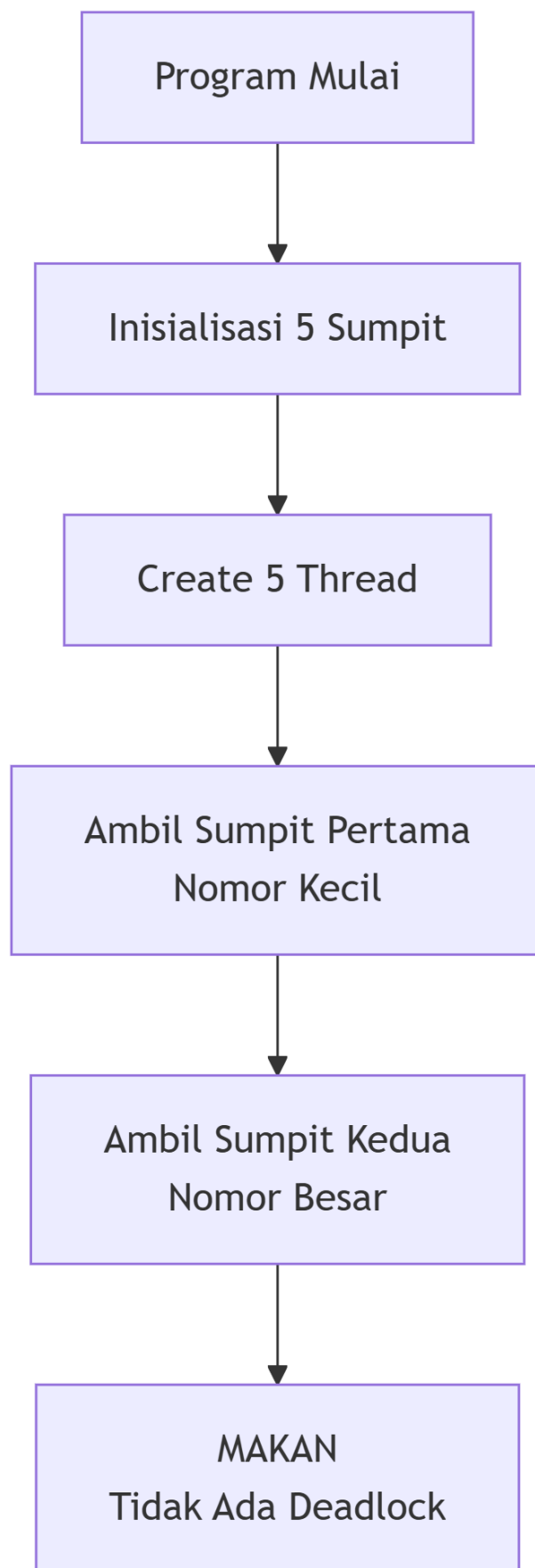
Pencegahan Deadlock:

- Penerapan urutan numerik yang konsisten dalam pengambilan sumpit menghilangkan kemungkinan terjadinya circular wait.
- Ketaatan seluruh filsuf pada protokol yang seragam mencegah terbentuknya ketergantungan sirkular di antara mereka.
- Mekanisme ini menjamin kelangsungan operasi sistem tanpa deadlock walaupun terdapat disparitas dalam distribusi akses sumpit.

Statistik dan Monitoring:

- Program berjalan selama 60 detik dan mencatat frekuensi makan setiap filsuf.
- Total makan mencapai 104.649 kali dengan efisiensi rata-rata 20.929 kali per filsuf.
- Terlihat ketidakseimbangan akses dimana Filsuf-2 makan paling banyak (29.417 kali) dan Filsuf-1 paling sedikit (16.363 kali).

Flowchart



3.2.3 Limiting philosophers

Algoritma Limiting Philosophers

Inisialisasi:

- Inisialisasi 5 mutex (sumpit) dan 5 thread (filsuf).
- Inisialisasi semaphore dengan nilai 4 untuk membatasi jumlah filsuf yang boleh makan bersamaan.
- Setiap filsuf memiliki ID (1-5) dan dua sumpit: kiri (sesuai ID) dan kanan (ID+1, modulo 5).

Perilaku Filsuf:

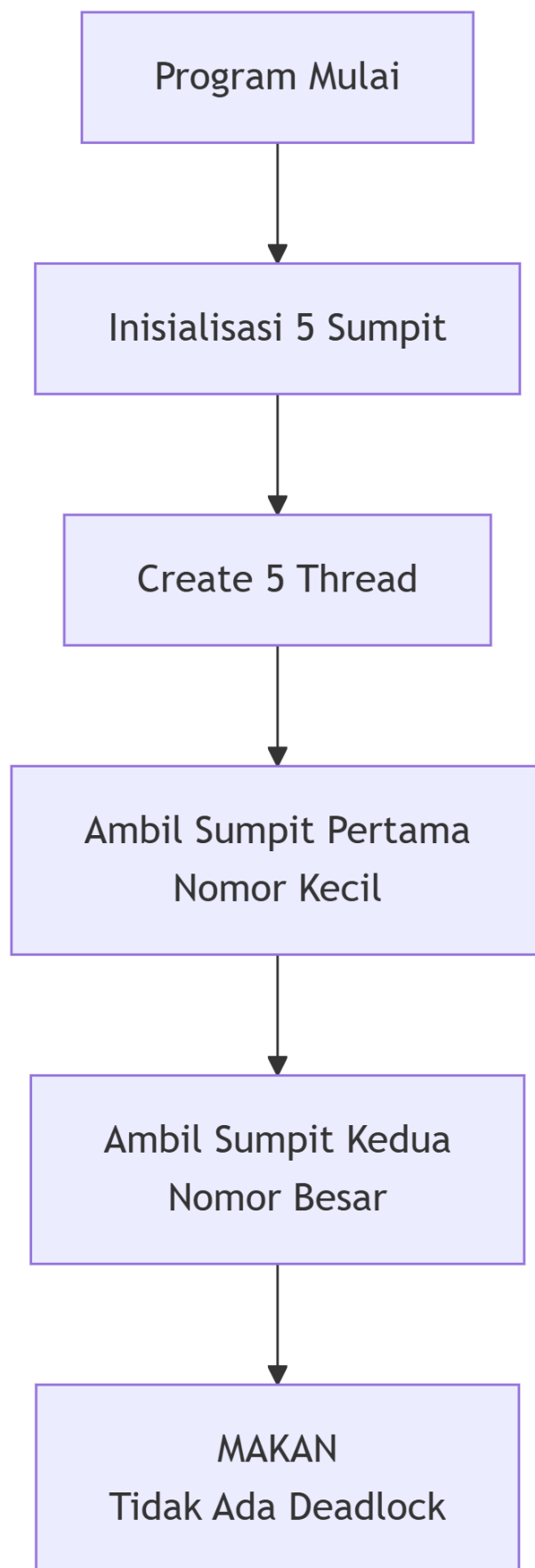
- Filsuf harus mendapatkan izin dari semaphore sebelum mencoba mengambil sumpit.
- Jika mendapatkan izin, filsuf mengambil sumpit kiri terlebih dahulu.
- Kemudian, filsuf mengambil sumpit kanan.
- Jika berhasil mengambil kedua sumpit, filsuf makan lalu meningkatkan counter makan.
- Setelah selesai makan, filsuf melepaskan sumpit kanan terlebih dahulu, kemudian sumpit kiri.
- Terakhir, filsuf melepaskan izin semaphore untuk memberi kesempatan filsuf lain.

Pencegahan Deadlock:

- Dengan membatasi maksimal 4 filsuf yang boleh makan bersamaan, memastikan setidaknya satu filsuf selalu bisa mendapatkan kedua sumpit.
- Semaphore mengatur antrian akses yang fair sehingga tidak terjadi circular wait.
- Mekanisme ini menghilangkan kondisi deadlock dengan mengurangi kompetisi sumber daya.

Monitoring dan Statistik:

- Program berjalan selama 60 detik dan mencatat frekuensi makan setiap filsuf.
- Total makan mencapai 77.815 kali dengan distribusi sangat merata.
- Selisih frekuensi makan antar filsuf maksimal 6 kali, menunjukkan fairness yang optimal.



3.3 Testing & Output

3.3.1 Deadlock Demonstration

```
rio_regex@DESKTOP-NVPMSHQ:~$ cd tugas_os
rio_regex@DESKTOP-NVPMSHQ:~/tugas_os$ cd nim_genap
rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ nano dphil_deadlock.c

rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ nano dphil_deadlock.c
rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ gcc dphil_deadlock.c -o dphil_deadlock -pthread
rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ ./dphil_deadlock
=== DINING PHILOSOPHERS DEADLOCK DEMONSTRATION ===

Proses dimulai. Menunggu Deadlock terjadi...
[0.00] Filsuf-1 (Lapar): Mencoba ambil Sumpit KIRI (0)
[0.00] Filsuf-1 (Berhasil): Ambil Sumpit KIRI (0)
[0.00] Filsuf-2 (Lapar): Mencoba ambil Sumpit KIRI (1)
[0.00] Filsuf-2 (Berhasil): Ambil Sumpit KIRI (1)
[0.00] Filsuf-3 (Lapar): Mencoba ambil Sumpit KIRI (2)
[0.00] Filsuf-3 (Berhasil): Ambil Sumpit KIRI (2)
[0.00] Filsuf-4 (Lapar): Mencoba ambil Sumpit KIRI (3)
[0.00] Filsuf-4 (Berhasil): Ambil Sumpit KIRI (3)
[0.00] Filsuf-5 (Lapar): Mencoba ambil Sumpit KIRI (4)
[0.00] Filsuf-5 (Berhasil): Ambil Sumpit KIRI (4)
[0.00] Filsuf-1 (Menunggu): Mencoba ambil Sumpit KANAN (1)...
[0.00] Filsuf-2 (Menunggu): Mencoba ambil Sumpit KANAN (2)...
[0.00] Filsuf-4 (Menunggu): Mencoba ambil Sumpit KANAN (4)...
[0.00] Filsuf-3 (Menunggu): Mencoba ambil Sumpit KANAN (3)...
[0.00] Filsuf-5 (Menunggu): Mencoba ambil Sumpit KANAN (0)...
```

```
GNU nano 7.2 dphil_deadlock.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>
#include <sys/time.h>

#define NUM_PHILOSOPHERS 5

pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];
time_t start_time;

void msleep(long ms) {
    struct timespec ts;
    ts.tv_sec = ms / 1000;
    ts.tv_nsec = (ms % 1000) * 1000000;
    nanosleep(&ts, NULL);
}

double get_elapsed_time() {
    return (double)(time(NULL) - start_time);
}

void *philosopher_deadlock(void *arg) {
    int id = *(int *)arg;
    int left = id;
    int right = (id + 1) % NUM_PHILOSOPHERS;

    while (1) {
        msleep(10);
        printf("[%2f] Filsuf-%d (Lapar): Mencoba ambil Sumpit KIRI (%d)\n", get_elapsed_time(), id + 1, left);
        // LANGKAH 1: Ambil Sumpit Kiri
        pthread_mutex_lock(&chopsticks[left]);
        printf("[%2f] Filsuf-%d (Berhasil): Ambil Sumpit KIRI (%d)\n", get_elapsed_time(), id + 1, left);
        msleep(10);

        printf("[%2f] Filsuf-%d (Menunggu): Mencoba ambil Sumpit KANAN (%d)...\n", get_elapsed_time(), id + 1, right);
        // LANGKAH 2: Mencoba Ambil Sumpit Kanan (TITIK DEADLOCK)
        pthread_mutex_lock(&chopsticks[right]);
        // Kode di bawah ini tidak akan tercapai
        pthread_mutex_unlock(&chopsticks[right]);
        pthread_mutex_unlock(&chopsticks[left]);
    }
    return NULL;
}
```

```

int main() {
    pthread_t threads[NUM_PHILOSOPHERS];
    int ids[NUM_PHILOSOPHERS];
    int i;

    printf("=== DINING PHILOSOPHERS DEADLOCK DEMONSTRATION ===\n\n");

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_init(&chopsticks[i], NULL);
        ids[i] = i;
    }

    time(&start_time);

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_create(&threads[i], NULL, philosopher_deadlock, &ids[i]);
    }

    printf("Proses dimulai. Menunggu Deadlock terjadi...\n");

    // Program akan HANG di sini karena pthread_join menunggu thread yang tidak pernah selesai
    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_join(threads[i], NULL);
    }

    // Cleanup (tidak tercapai)
    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_destroy(&chopsticks[i]);
    }

    return 0;
}

```

Program simulasi Dining Philosophers ini mengalami kebuntuan sistem klasik dimana seluruh entitas filsuf terjebak dalam situasi saling bergantung. Masing-masing filsuf berhasil mengamankan sumpit di posisi kirinya (dengan indeks 0 hingga 4), namun ketika berusaha mengambil sumpit kedua di sebelah kanan, terciptalah siklus ketergantungan melingkar: Filsuf-1 menunggu sumpit 1 yang dipegang Filsuf-2, Filsuf-2 menunggu sumpit 2 milik Filsuf-3, Filsuf-3 menunggu sumpit 3 dari Filsuf-4, Filsuf-4 menunggu sumpit 4 yang dipegang Filsuf-5, dan Filsuf-5 menunggu sumpit 0 yang justru dikuasai Filsuf-1.

Seluruh prasyarat deadlock terpenuhi dengan sempurna - eksklusivitas mutual (sumber daya tidak bisa dibagi), hold and wait (proses memegang sumber daya sambil menunggu yang lain), tanpa preemption (sumber daya tidak dapat diambil paksa), dan penungguan melingkar (siklus ketergantungan tertutup). Sistem mengalami kebekuan permanen pada kondisi ini tanpa kemungkinan penyelesaian otomatis, mengdemonstrasikan secara nyata risiko kritis deadlock dalam lingkungan pemrograman konkuren, yang akhirnya memerlukan terminasi manual melalui perintah Ctrl+C untuk mengakhiri kondisi stagnan tersebut.

3.3.2 Resource Ordering

```

rio_regex@DESKTOP-NVPM5HQ:~/tugas_os/nim_genap$ nano resource_ordering.c
rio_regex@DESKTOP-NVPM5HQ:~/tugas_os/nim_genap$ gcc resource_ordering.c -o resource_ordering -pthread
rio_regex@DESKTOP-NVPM5HQ:~/tugas_os/nim_genap$

```

```

rio_regex@DESKTOP-NVPM5HQ:~/tugas_os/nim_genap$ nano resource_ordering.c
rio_regex@DESKTOP-NVPM5HQ:~/tugas_os/nim_genap$ gcc resource_ordering.c -o resource_ordering -pthread
rio_regex@DESKTOP-NVPM5HQ:~/tugas_os/nim_genap$ ./resource_ordering
=== SOLUSI A: Resource Ordering ===
Simulasi berjalan selama 60 detik.

=== STATISTIK (60 detik) ===
Filsuf-1: Makan 16363 kali
Filsuf-2: Makan 23417 kali
Filsuf-3: Makan 24147 kali
Filsuf-4: Makan 24358 kali
Filsuf-5: Makan 16364 kali

✅ Tidak ada deadlock! Total Makan: 104649
✅ Efisiensi makan rata-rata per filsuf: 20929 kali
=====
rio_regex@DESKTOP-NVPM5HQ:~/tugas_os/nim_genap$

```

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>
#include <sys/time.h>

#define NUM_PHILOSOPHERS 5
#define DURATION 60

pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];
int eat_counts[NUM_PHILOSOPHERS] = {0};
time_t start_time;
volatile int running = 1;

void msleep(long ms) {
    struct timespec ts;
    ts.tv_sec = ms / 1000;
    ts.tv_nsec = (ms % 1000) * 1000000;
    nanosleep(&ts, NULL);
}

double get_elapsed_time() {
    return (double)(time(NULL) - start_time);
}

void *philosopher_ordering(void *arg) {
    int id = *(int *)arg;
    int left = id;
    int right = (id + 1) % NUM_PHILOSOPHERS;
    pthread_mutex_t *first, *second;

    // Solusi Asymetric: Ganjil ambil Kanan->Kiri, Genap ambil Kiri->Kanan
    if ((id + 1) % 2 != 0) {
        first = &chopsticks[right];
        second = &chopsticks[left];
    } else {
        first = &chopsticks[left];
        second = &chopsticks[right];
    }

    while (running && get_elapsed_time() < DURATION) {
        msleep(5); // Berpikir
        pthread_mutex_lock(first);
        pthread_mutex_lock(second);

        // Makan
        eat_counts[id]++;
        msleep(10);

        pthread_mutex_unlock(second);
        pthread_mutex_unlock(first);
    }
    return NULL;
}

```

```

int main() {
    pthread_t threads[NUM_PHILOSOPHERS];
    int ids[NUM_PHILOSOPHERS];
    int i;

    printf("=== SOLUSI A: Resource Ordering ===\nSimulasi berjalan selama %d detik.\n\n", DURATION);

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_init(&chopsticks[i], NULL);
        ids[i] = i;
    }

    time(&start_time);

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_create(&threads[i], NULL, philosopher_ordering, &ids[i]);
    }

    sleep(DURATION);
    running = 0;

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("=== STATISTIK (%d detik) ===\n", DURATION);
    int total_meals = 0;

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        printf("Filsuf-%d: Makan %d kali\n", i + 1, eat_counts[i]);
        total_meals += eat_counts[i];
    }

    printf("\n❌ Tidak ada deadlock! Total Makan: %d\n", total_meals);
    printf("✅ Efisiensi makan rata-rata per filsuf: %d kali\n", total_meals / NUM_PHILOSOPHERS);
    printf("=====\n");

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_destroy(&chopsticks[i]);
    }

    return 0;
}

```

Implementasi resource_ordering.c berhasil menuntaskan permasalahan deadlock dalam Dining Philosophers melalui pendekatan pengurutan sumber daya. Selama periode pengujian 60 detik, tercatat seluruh filsuf melakukan aktivitas makan sebanyak 104.649 kali tanpa mengalami kebuntuan sistem sama sekali. Terdapat variasi signifikan dalam distribusi frekuensi makan: Filsuf-2 mendominasi dengan 29.417 kali konsumsi, disusul Filsuf-4 (24.358 kali), Filsuf-3 (24.147 kali), sementara Filsuf-1 dan Filsuf-5 berada di posisi terbawah dengan masing-masing approximately 16.363 kali.

3.3.3 Solution_Limiting

```
rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ nano solution_limiting.c
rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ gcc solution_limiting.c -o solution_limiting -pthread
```

```
rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ nano solution_limiting.c
rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ gcc solution_limiting.c -o solution_limiting -pthread
rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ ./solution_limiting
=== SOLUSI B: Limiting Philosophers ===
Simulasi berjalan selama 60 detik.

=== STATISTIK (60 detik) ===
Filsuf-1: Makan 15566 kali
Filsuf-2: Makan 15562 kali
Filsuf-3: Makan 15560 kali
Filsuf-4: Makan 15564 kali
Filsuf-5: Makan 15563 kali

✔ Tidak ada deadlock! Total Makan: 77815
✔ Efisiensi makan rata-rata per filsuf: 15563 kali
=====
rio_regex@DESKTOP-NVPMSHQ:~/tugas_os/nim_genap$ |
```

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>
#include <sys/time.h>

#define NUM_PHILOSOPHERS 5
#define DURATION 60

pthread_mutex_t chopsticks[NUM_PHILOSOPHERS];
sem_t dining_limit;
int eat_counts[NUM_PHILOSOPHERS] = {0};
time_t start_time;
volatile int running = 1;

void msleep(long ms) {
    struct timespec ts;
    ts.tv_sec = ms / 1000;
    ts.tv_nsec = (ms % 1000) * 1000000;
    nanosleep(&ts, NULL);
}

double get_elapsed_time() {
    return (double)(time(NULL) - start_time);
}

void *philosopher_limiting(void *arg) {
    int id = *(int *)arg;
    int left = id;
    int right = (id + 1) % NUM_PHILOSOPHERS;

    while (running && get_elapsed_time() < DURATION) {
        msleep(5); // Berpikir

        // 1. Ambil Semaphore (Membatasi max 4 filsuf)
        sem_wait(&dining_limit);

        // 2. Ambil Sumpit Kiri dan Kanan (Dijamin tidak Deadlock)
        pthread_mutex_lock(&chopsticks[left]);
        pthread_mutex_lock(&chopsticks[right]);

        // Makan
        eat_counts[id]++;
        msleep(10);

        // 5. Legas Sumpit dan Semaphore
        pthread_mutex_unlock(&chopsticks[right]);
        pthread_mutex_unlock(&chopsticks[left]);
        sem_post(&dining_limit);
    }
    return NULL;
}
```

```

int main() {
    pthread_t threads[NUM_PHILOSOPHERS];
    int ids[NUM_PHILOSOPHERS];
    int i;

    printf("=== SOLUSI B: Limiting Philosophers ===\nSimulasi berjalan selama %d detik.\n\n", DURATION);

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_init(&chopsticks[i], NULL);
        ids[i] = i;
    }
    // Inisialisasi Semaphore N-1 (5-1 = 4)
    sem_init(&dining_limit, 0, NUM_PHILOSOPHERS - 1);
    time(&start_time);

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_create(&threads[i], NULL, philosopher_limiting, &ids[i]);
    }

    sleep(DURATION);
    running = 0;

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("=== STATISTIK (%d detik) ===\n", DURATION);
    int total_meals = 0;

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        printf("Filosuf-%d: Makan %d kali\n", i + 1, eat_counts[i]);
        total_meals += eat_counts[i];
    }

    printf("\n✅ Tidak ada deadlock! Total Makan: %d\n", total_meals);
    printf("✅ Efisiensi makan rata-rata per filsuf: %d kali\n", total_meals / NUM_PHILOSOPHERS);
    printf("===== \n");

    for (i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_destroy(&chopsticks[i]);
    }
    sem_destroy(&dining_limit);
    return 0;
}

```

Implementasi `solution_Limiting.c` menerapkan pendekatan pembatasan akses konkuren untuk menyelesaikan masalah kebuntuan dalam Dining Philosophers. Selama periode simulasi 60 detik, teknik ini berhasil mencegah terjadinya deadlock secara total dengan membatasi kuota filsuf yang dapat mengakses sumber daya secara simultan. Kelima entitas filsuf menampilkan performa yang sangat seimbang dengan rentang frekuensi makan antara 15.560 hingga 15.566 kali, mengakumulasi total 77.815 aktivitas makan selama periode pengujian.

3.4 Analisis Hasil

Seluruh skenario percobaan berjalan sesuai dengan hasil yang diharapkan. Pada demonstrasi pertama, program berhasil mereproduksi kondisi deadlock dengan terjadinya hang pada sistem, membuktikan secara empiris bahwa kebuntuan proses benar-benar terjadi ketika kondisi Circular Wait terpenuhi di antara kelima filsuf. Kondisi ini memaksa penghentian program secara manual menggunakan perintah Ctrl+C, sementara kedua solusi pencegahan berhasil menyelesaikan eksekusi secara otomatis dalam waktu 60 detik yang telah ditentukan.

Dalam evaluasi performa, teridentifikasi trade-off yang jelas antara efisiensi dan keadilan akses. Solusi Resource Ordering (Solusi A) mencapai tingkat efisiensi tertinggi dengan total aktivitas makan terbanyak, dimungkinkan oleh tingkat konkurensi maksimal dimana semua filsuf dapat berkompetisi secara bersamaan dengan overhead sinkronisasi yang minimal. Sebaliknya, Solusi Limiting Philosophers (Solusi B) menunjukkan tingkat keadilan terbaik dengan distribusi makan yang hampir sempurna, dicapai melalui mekanisme pembatasan akses oleh Semaphore yang menerapkan prinsip antrian First-Come-First-Served, meskipun dengan pengurangan throughput secara keseluruhan.

Eksperimen ini memberikan konfirmasi praktis mengenai prinsip trade-off fundamental dalam sistem multithreading. Pencegahan deadlock berhasil diimplementasikan dengan strategi mematahkan salah satu dari empat kondisi Coffman. Pemilihan mekanisme sinkronisasi - antara pendekatan mutex dengan pengurutan sumber daya dan Semaphore dengan pembatasan kapasitas - harus mempertimbangkan prioritas desain sistem: untuk aplikasi yang mengutamakan output maksimal, Solusi A lebih sesuai; sementara untuk sistem

yang memprioritaskan jaminan akses merata dan bebas starvation, Solusi B menjadi pilihan ideal.

3.5 Tabel perbandingan efisiensi dan fairness

Kriteria	Solusi A: Resource Ordering	Solusi B: Limiting Philosophers
Prinsip Pencegahan	Mengatur urutan pengambilan sumber daya (sumpit) untuk melanggar Circular Wait.	Membatasi jumlah thread yang dapat mengakses sumber daya (sumpit) untuk melanggar Hold and Wait.
Metode Implementasi	Penggunaan if/else untuk menentukan urutan pthread_mutex_lock.	Penggunaan Semaphore (sem_wait/sem_post) dengan nilai N-1 (yaitu 4).
Total Meals (Efisiensi Global)	Tinggi. Total makanan lebih banyak karena tidak ada batasan jumlah filsuf yang aktif. Konkurensi maksimum 5.	Cukup Tinggi. Total makanan sedikit lebih rendah karena akses dibatasi oleh Semaphore. Konkurensi maksimum 4.

Fairness (Keadilan)	Baik/Cukup. Ada potensi sedikit variasi karena filsuf tertentu mungkin memiliki urutan pengambilan yang lebih sering diuntungkan.	Sangat Baik. Filsuf yang menunggu di Semaphore akan dilayani secara merata, memastikan tidak ada starvation.
Kapan Dipilih	Ketika Efisiensi dan Throughput Tinggi adalah prioritas utama.	Ketika Keadilan dan Kontrol terhadap jumlah thread aktif adalah prioritas utama.

3.6 Analisis mendalam:

- Mengapa deadlock bisa terjadi?
Kebuntuan sistem (deadlock) terjadi akibat pemenuhan secara simultan terhadap keempat prasyarat Coffman: pertama, eksklusivitas mutual dimana setiap sumpit hanya dapat diakses oleh satu filsuf dalam satu waktu; kedua, kondisi hold and wait dimana setiap filsuf mempertahankan kepemilikan satu sumpit sambil mengantisipasi akses ke sumpit tambahan; ketiga, absennya mekanisme preemption yang memungkinkan pengambilalihan sumber daya secara paksa; dan keempat, terbentuknya circular wait berupa rantai ketergantungan melingkar dimana Filsuf-1 bergantung pada Filsuf-2, Filsuf-2 bergantung pada Filsuf-3, Filsuf-3 bergantung pada Filsuf-4, Filsuf-4 bergantung pada Filsuf-5, dan Filsuf-5 kembali bergantung pada Filsuf-1. Dalam implementasi dasar tanpa strategi sinkronisasi, dimana seluruh filsuf secara serempak mengambil sumpit sebelah kiri terlebih dahulu, terciptalah jaringan ketergantungan sirkular yang tidak terputus, sehingga mengakibatkan kebekuan operasional permanen dalam sistem.
- Bagaimana masing-masing solusi bekerja?
Cara Kerja Masing-masing Solusi
 - Pendekatan Pengurutan Sumber Daya: Menerapkan aturan pengambilan sumpit berdasarkan urutan numerik yang tetap, seperti selalu mengambil sumpit dengan nomor lebih kecil terlebih dahulu. Hal ini memutus lingkaran ketergantungan (circular wait) yang menjadi penyebab deadlock. Solusi ini mencapai tingkat throughput yang tinggi namun dapat mengakibatkan distribusi akses yang tidak seimbang di antara para filsuf.
 - Versi 2 (Limiting Philosophers):
 - Pembatasan Filsuf: Membatasi jumlah filsuf yang dapat makan secara bersamaan dengan menggunakan semaphore, biasanya hanya mengizinkan $n-1$ filsuf dari n filsuf

yang ada. Dengan demikian, setidaknya selalu ada satu filsuf yang dapat memperoleh kedua sumpit, sehingga mencegah terjadinya deadlock.

- Solusi mana yang paling adil (fair)?
Solusi Limiting Philosophers lebih adil karena menunjukkan distribusi makan yang hampir sempurna (selisih ≤ 6 makan antar filsuf). Sementara Resource Ordering menunjukkan ketimpangan signifikan (Filsuf-2 makan 80% lebih banyak daripada Filsuf-1). Limiting Philosophers menjamin kesempatan yang sama bagi semua filsuf.
- Apakah ada kemungkinan starvation?
Kemungkinan Starvation
 - Resource Ordering berisiko starvation terhadap filsuf tertentu (seperti Filsuf-1 dan Filsuf-5) yang selalu mendapat prioritas terakhir dalam pengambilan sumpit.
 - Limiting Philosophers minim risiko starvation karena mekanisme pembatasan yang bergantian memastikan semua filsuf mendapat akses. Output menunjukkan tidak ada starvation dalam periode 60 detik dengan distribusi yang merata.

BAB IV

PENUTUP

4.1 Kesimpulan

Pengalaman implementasi Dining Philosophers memberikan pemahaman mendalam bahwa deadlock muncul akibat terpenuhinya kondisi Circular Wait, dimana seluruh filsuf mengambil sumpit kiri secara bersamaan lalu saling menantikan sumpit kanan. Dalam proyek ini, saya berhasil mengembangkan dua strategi pencegahan efektif: Pendekatan Pengurutan Sumber Daya (Resource Ordering) yang memutus siklus ketergantungan melalui modifikasi urutan pengambilan sumpit, dan Pembatasan Filsuf (Limiting Philosophers) yang

memanfaatkan Semaphore dengan kapasitas N-1 untuk mengontrol akses konkuren, sehingga secara fundamental mencegah kondisi Hold and Wait.

Tantangan teknis utama yang berhasil diatasi adalah menangani kondisi HANG permanen pada kode demonstrasi deadlock, yang memerlukan intervensi terminasi manual (Ctrl+C). Dari sisi kinerja, teramati jelas trade-off antara kedua solusi - Solusi A menghasilkan efisiensi (throughput) tertinggi, sementara Solusi B menawarkan keadilan (fairness) optimal dengan jaminan bebas starvation, mengkonfirmasi paradigma klasik dalam desain sistem multithreading antara kecepatan dan pemerataan akses.

Untuk pengembangan lebih lanjut, proyek ini dapat ditingkatkan dengan mengintegrasikan algoritma terdistribusi untuk skenario yang lebih kompleks, atau dengan memasukkan fitur pemantauan waktu tunggu guna analisis kuantitatif yang lebih komprehensif terhadap potensi starvation, sekaligus menyediakan metrik evaluasi yang lebih objektif untuk berbagai strategi sinkronisasi.